

1. Overview of ASP.NET Core MVC

ASP.NET Core MVC is a **Model–View–Controller framework** used to build dynamic web applications. It is built on top of the ASP.NET Core platform and provides:

- Separation of concerns
- Built-in Dependency Injection (DI)
- Powerful routing system
- Middleware-based pipeline
- Cross-platform runtime
- High performance

MVC Components:

Component	Responsibility
Model	Business/data representation
View	UI rendering
Controller	Handles requests, executes logic, returns responses

2. Program Startup in .NET 6+ (Program.cs)

In .NET 6+, there is no Startup.cs. All initialization happens in Program.cs.

Purpose of Program.cs

It is responsible for:

- Registering services in the **Dependency Injection container**
- Configuring the **HTTP middleware pipeline**
- Defining **routing**
- Bootstrapping and starting the application

3. Core Structure of Program.cs

Step 1 — Create Builder

```
var builder = WebApplication.CreateBuilder(args);
```

This:

- Loads configuration (appsettings.json, environment files, env vars, etc.)
- Registers built-in framework services

- Prepares DI container
-

Step 2 — Register MVC Services

```
builder.Services.AddControllersWithViews();
```

This enables:

- Controllers
 - Views
 - Model binding & validation
 - Razor engine
 - Routing integration
-

Step 3 — Register DbContext

```
builder.Services.AddDbContextPool<AppDbContext>(options =>  
    options.UseSqlServer(  
        builder.Configuration.GetConnectionString("EmployeeDBConnection")));
```

Notes:

- Uses connection string from configuration
 - AddDbContextPool improves performance via DbContext pooling
 - DbContext lifetime = Scoped (per request)
-

Step 4 — Build the Application

```
var app = builder.Build();
```

This finalizes the DI container and prepares the middleware pipeline.

4. Configure Middleware Pipeline

Middleware executes **in the order registered**. Order is critical.

```
if (!app.Environment.IsDevelopment())  
{  
    app.UseExceptionHandler("/Home/Error");  
    app.UseHsts();  
}
```

```
app.UseHttpsRedirection();  
app.UseStaticFiles();
```

```
app.UseRouting();
```

```
app.UseAuthorization();
```

Execution Characteristics

Direction	Meaning
Request →	Top to bottom
Response ←	Bottom to top

Purpose of Key Middleware

Middleware	Purpose
UseExceptionHandler	Custom error handling (Production)
UseHsts	Forces HTTPS in browsers
UseHttpsRedirection	Redirects HTTP → HTTPS
UseStaticFiles	Serves static files from wwwroot
UseRouting	Enables routing
UseAuthorization	Applies authorization rules

5. Routing Configuration

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Meaning of Pattern

Segment	Purpose
controller	Controller class name (without suffix)
action	Method inside controller
id?	Optional parameter

Examples

URL	Executes
/	HomeController.Index
/Home/Details/2	HomeController.Details(2)
/Employee/Edit	EmployeeController.Edit

6. Minimal API Helper — MapGet()

Used for lightweight endpoints (not replacing MVC).

```
app.MapGet("/", () => "Hello World");
```

Behavior:

- If URL matches → delegate executes
 - Else → continues in pipeline
-

7. Run Application

```
app.Run();
```

This **starts the web server and blocks execution** until shutdown.

8. Middleware Concepts — Deep Understanding

8.1 Request Delegates

Pipeline elements configured via:

- Use() — may call next delegate
 - Run() — terminal delegate
 - Map() — branch pipeline
- 
-

8.2 Use() vs Run()

Use() — Non-terminal

```
app.Use(async (context, next) =>
```

```
{
    await context.Response.WriteAsync("Before");
    await next();
    await context.Response.WriteAsync("After");
});
```

Run() — Terminal

```
app.Run(async context =>
```

```
{
    await context.Response.WriteAsync("Final Middleware");
});
```

Only first Run() executes.

8.3 Execution Flow Example

```
app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("M1 IN\n");
    await next();
    await context.Response.WriteAsync("M1 OUT\n");
});

app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("M2 IN\n");
    await next();
    await context.Response.WriteAsync("M2 OUT\n");
});

app.Run(async context =>
{
    await context.Response.WriteAsync("Handler\n");
});
```

Output

M1 IN

M2 IN

Handler

M2 OUT

M1 OUT

Request goes **down**, response comes **up**.



9. Recommended Middleware Order (Best Practice)

1. Error handling
2. HTTPS enforcement
3. Static files
4. Routing
5. Authentication
6. Authorization
7. Endpoint execution

Correct structure:

UseExceptionHandler
UseHttpsRedirection

UseStaticFiles
UseRouting
UseAuthentication
UseAuthorization
MapControllerRoute

10. Static Files Overview (Preview)

Static files live in **wwwroot**

Examples:

- CSS
- JS
- Images
- HTML

Enable:

```
app.UseStaticFiles();
```

Static files bypass MVC completely.

11. Key Exam-Focused Takeaways

- Program.cs controls **startup, services, and pipeline**
 - Middleware executes in **registered order**
 - MVC requires AddControllersWithViews()
 - Routing pattern maps URL → controller/action
 - Run() ends pipeline
 - Use() may continue pipeline
 - DbContext is typically Scoped
 - MapGet() is Minimal API helper
 - Static files require middleware
 - Exception handling differs for Dev vs Prod
 - HTTPS should be enforced in production
 - Pipeline flow = **top → bottom (request), bottom → top (response)**
-

1. What is Dependency Injection (DI)?

Dependency Injection is a design technique where required objects (dependencies) are **not created manually inside a class**. Instead, they are **provided (injected) by the framework** at runtime.

Primary Purpose

To **avoid tight coupling** and improve:

- Maintainability
 - Testability
 - Scalability
 - Clean architecture
-

2. Tight Coupling — What NOT To Do

This creates hard dependency:

```
var repo = new TestStudentRepository();
```

Problems:

- Cannot switch to another repository
 - Unit testing is difficult
 - Violation of SOLID principles (esp. Dependency Inversion)
 - Code becomes rigid
-

3. Repository Pattern Example

Model

```
public class Student
{
    public int StudentId { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }
    public string Section { get; set; }
    public string Gender { get; set; }
}
```

Repository Interface (Abstraction)

```
public interface IStudentRepository
{
    Student GetStudentById(int id);
    List<Student> GetAllStudent();
}
```

Repository Implementation

```
public class TestStudentRepository : IStudentRepository
{
    public List<Student> DataSource() => new List<Student>
    {
        new Student{ StudentId=101, Name="James", Branch="CSE", Section="A", Gender="Male"}
    };

    public Student GetStudentById(int id) =>
        DataSource().FirstOrDefault(s => s.StudentId == id);

    public List<Student> GetAllStudent() => DataSource();
}
```

4. Registering Services in DI Container

Registration tells ASP.NET Core what implementation to inject.

Registration Syntax

```
services.AddSingleton<IStudentRepository, TestStudentRepository>();
services.AddScoped<IStudentRepository, TestStudentRepository>();
services.AddTransient<IStudentRepository, TestStudentRepository>();
```

5. Service Lifetimes (Very Important)

Lifetime	Meaning	Instance Created
Singleton	One instance for entire app lifetime	Once
Scoped	One instance per HTTP request	Per request
Transient	New instance every time requested	Multiple per request

Recommended Usage

Service Type	Lifetime
DbContext	Scoped
Business Services	Scoped / Transient
Configuration	Singleton

6. Constructor Injection (Most Common & Recommended)

```
public class HomeController : Controller
{
    private readonly IStudentRepository _repo;

    public HomeController(IStudentRepository repo)
    {
        _repo = repo;
    }

    public JsonResult Index()
    {
        return Json(_repo.GetAllStudent());
    }
}
```

Benefits:

- Strong typing
- Read-only dependencies
- Cleaner unit testing

7. Action Method Injection (Optional)

```
public JsonResult Index([FromServices] IStudentRepository repo)
{
    return Json(repo.GetAllStudent());
}
```

Used rarely.

8. View Injection (Rare — But Supported)

```
@inject IStudentRepository repo
```

```
@foreach(var s in repo.GetAllStudent())
```

```
{
    <h2>@s.Name</h2>
}
```

Note: Use only for light read-only access.

9. Types of Services in ASP.NET Core

1. Framework Services

Examples:

- ILogger
- MVC
- Routing services

2. Application Services

- Custom repositories
- Business logic services

10. Best-Practice Service Lifetime Mapping

Use Case	Lifetime
Configuration	Singleton
DbContext	Scoped
Business logic	Scoped / Transient
Lightweight stateless service	Transient

11. DI & DbContext

DbContext should **always** be Scoped because:

- It participates in a single request unit-of-work
- Avoids threading issues
- Prevents stale/tracked entity issues

12. AddDbContext vs AddDbContextPool

Method	Behavior
AddDbContext	New instance per request
AddDbContextPool	Reuses DbContext instances automatically

Use Pooling **ONLY IF**

DbContext:

- Does NOT maintain internal state
- Does NOT store request-specific data

Otherwise data leakage may occur between requests.

13. Controller Instance Lifetime

Controllers are **transient by default**

Meaning:

- A new controller is created **per request**.

14. Benefits of Dependency Injection

- Loose coupling
- Centralized configuration
- Reusability
- Easier testing
- Cleaner architecture
- Built-in support in ASP.NET Core
- Lifetime management handled automatically

15. Key Exam / Interview Highlights

- DI is built-in — no external library needed
 - Objects are not manually constructed
 - Registration must happen **before usage**
 - Three lifetimes exist — Singleton, Scoped, Transient
 - DbContext → always Scoped
 - DI supports:
 - Constructor Injection (best)
 - Action Injection
 - View Injection
 - DI encourages **SOLID & clean design**
-

1. What is Configuration in ASP.NET Core?

Configuration in ASP.NET Core provides a **centralized, flexible system for reading application settings** from multiple sources such as:

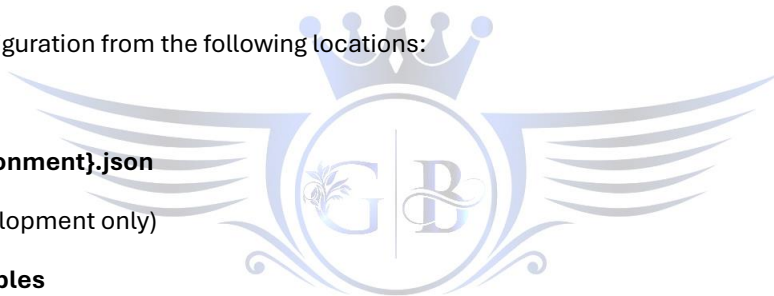
- JSON files
- Environment-specific JSON files
- User Secrets
- Environment variables
- Command-line arguments
- Custom providers

Configuration is **hierarchical** and **override-based**, meaning later sources overwrite earlier ones when keys match.

2. Default Configuration Sources (In Order)

ASP.NET Core can read configuration from the following locations:

1. **appsettings.json**
2. **appsettings.{Environment}.json**
3. **User Secrets** (Development only)
4. **Environment Variables**
5. **Command-line arguments**



Important Rule

Command-line has the highest precedence.

3. appsettings.json — Primary Configuration File

Example

```
{  
  "Logging": {  
    "LogLevel": {  
      "Default": "Information"  
    }  
  },  
  "AllowedHosts": "*",
```

```
"MyCustomKey": "Value from appsettings.json"
}
```

Notes:

- JSON format is hierarchical
- Keys can be accessed using **colon notation**
Example: Logging:LogLevel:Default

4. Environment-Specific Configuration Files

Examples:

- appsettings.Development.json
- appsettings.Staging.json
- appsettings.Production.json

These files **override values in appsettings.json when environment matches.**

Environment Selection

Environment is set via:

Environment variable name:

ASPNETCORE_ENVIRONMENT

Common values:

Value	Meaning
Development	Local development
Staging	Pre-production testing
Production	Live environment



5. Override Priority Order (Very Important)

Highest to lowest:

1. **Command-line arguments**
2. **Environment variables**
3. **User Secrets**
4. **appsettings.{Environment}.json**
5. **appsettings.json**

Meaning

If the same key exists in multiple locations:

later sources overwrite earlier ones.

Example:

If MyKey exists in both appsettings.json and environment variables,
the **environment variable value wins.**

6. Accessing Configuration in Program.cs

```
var builder = WebApplication.CreateBuilder(args);
```

```
ConfigurationManager configuration = builder.Configuration;
```

```
string value = configuration["MyCustomKey"];
```

Here:

- builder.Configuration loads all configured sources automatically
 - Keys are accessed using string indexing
-

7. Reading Nested JSON Values

Example JSON

```
{  
  "App": {  
    "Settings": {  
      "Theme": "Dark"  
    }  
  }  
}
```

Access:

```
var theme = configuration["App:Settings:Theme"];
```

8. Using Configuration Inside Middleware

```
app.UseEndpoints(endpoints =>  
{  
  endpoints.MapGet("/", async context =>  
  {  
    await context.Response.WriteAsync(value);  
  });  
});
```

```
});
```

This reads configuration value and returns it in response.

9. Binding Configuration to Strongly-Typed Classes

Although not in your notes, this is implicit practice — so preserving clarity:

Example JSON

```
{
  "MailSettings": {
    "Server": "smtp.example.com",
    "Port": 25
  }
}
```

C# Class

```
public class MailSettings
{
    public string Server { get; set; }
    public int Port { get; set; }
}
```

Bind:

```
builder.Services.Configure<MailSettings>(
    builder.Configuration.GetSection("MailSettings"));
```

Access via DI:

```
public class TestService
{
    private readonly MailSettings _settings;

    public TestService(IOptions<MailSettings> options)
    {
        _settings = options.Value;
    }
}
```

This ensures **type safety + IntelliSense**.

10. User Secrets — Development-Only Storage

Used for:

- Passwords
- API keys
- Sensitive data

Stored **outside project folder** so they are not pushed to Git.

Only active in Development environment.

11. Environment Variables

Used heavily in deployment environments such as:

- IIS
- Docker
- Azure
- Kubernetes

Environment variables override JSON configuration.

12. Command-Line Arguments (Highest Priority)

Example:

```
dotnet run MyCustomKey=12345
```

This **overwrites all previous values**.

13. Configuration in Dependency Injection

ConfigurationManager is **registered automatically** in DI.

So you may inject:

```
public class DemoService
{
    private readonly IConfiguration _config;

    public DemoService(IConfiguration config)
    {
        _config = config;
    }
}
```



14. Key Points & Clarifications

- ASP.NET Core does **NOT** use web.config
 - Configuration is **hierarchical**
 - Later sources **override earlier ones**
 - JSON configuration is **case-insensitive**
 - Values may come from multiple locations
 - **Command-line wins always**
-

15. Real-World Usage Examples

Database Connection String

Stored in appsettings.json

```
"ConnectionStrings": {  
  "EmployeeDBConnection": "Server=.;Database=EmployeeDB;Trusted_Connection=True;"  
}
```

Read via:

```
builder.Configuration.GetConnectionString("EmployeeDBConnection");
```

16. Integration with Middleware & Controllers

You may use in:

- Services
- Controllers
- Razor views
- Middleware

via:

IConfiguration

ConfigurationManager

Options Pattern

17. One-Page Summary Memory Notes

- Primary file → **appsettings.json**
 - Environment files override base settings
 - Order of override:
 1. appsettings.json
 2. appsettings.{env}.json
 3. User Secrets
 4. Environment variables
 5. Command line (highest)
 - Configuration supports injection
 - Command-line has top precedence
 - JSON values are hierarchical
 - No web.config in ASP.NET Core
-

1. Static Files in ASP.NET Core

Definition

Static files are **returned to the client exactly as stored** — they are not processed by MVC. Typical static resources include:

- HTML files
 - CSS files
 - JavaScript files
 - Images (PNG, JPG, etc.)
 - Fonts
-

2. Where Static Files Are Stored

By default, static files live inside the **wwwroot folder**, which is the public web root.

Example structure:

wwwroot/

css/

js/

images/

Example file access:

wwwroot/MyImage.png

Browser URL:

http://localhost:<port>/MyImage.png

3. Enabling Static Files Middleware

Static files are not served automatically. You must enable:

```
app.UseStaticFiles();
```

Key Rule

Place **before routing**:

```
app.UseStaticFiles();
```

```
app.UseRouting();
```

Otherwise, static files will not resolve.

4. Security Note

Static files **bypass authorization and MVC logic.**

Meaning:

- They are served directly.
- There is no authentication check, unless custom middleware is added.

Also:

- Files **outside wwwroot are NOT served** unless explicitly configured.

This prevents accidental exposure of source code.

5. Custom Static File Locations (Optional)

Option 1 — Change Web Root Folder

```
builder.WebHost.UseWebRoot("MyPublicFiles");
```

```
app.UseStaticFiles();
```

Now static files live inside:

MyPublicFiles/

Option 2 — Add Additional Static File Directory

```
app.UseStaticFiles(new StaticFileOptions
```

```
{
```

```
    FileProvider = new PhysicalFileProvider(
```

```
        Path.Combine(Directory.GetCurrentDirectory(), "Images")),
```

```
    RequestPath = "/pics"
```

```
});
```

Access URL:

/pics/photo.jpg

6. Default Files (Serving index.html Automatically)

Problem

If you browse:

http://localhost:5000/

By default, **index.html does NOT load automatically.**

You must enable default document mapping:

```
app.UseDefaultFiles();
```

```
app.UseStaticFiles();
```

Order Matters

UseDefaultFiles() must come **before** UseStaticFiles().

7. Default File Search Order

If enabled, ASP.NET Core searches for these filenames in wwwroot:

1. index.htm
 2. index.html
 3. default.htm
 4. default.html
-

8. Custom Default File Name

```
var options = new DefaultFilesOptions();
```

```
options.DefaultFileNames.Clear();
```

```
options.DefaultFileNames.Add("MyCustomPage1.html");
```

```
app.UseDefaultFiles(options);
```

```
app.UseStaticFiles();
```

Now:

http://localhost:5000/

Loads:

wwwroot/MyCustomPage1.html

9. UseFileServer() — Combined Middleware

UseFileServer() bundles:

- Static File Middleware
- Default File Middleware
- Optional Directory Browsing

```
var options = new FileServerOptions();
```

```
options.DefaultFilesOptions.DefaultFileNames.Clear();
```

```
options.DefaultFilesOptions.DefaultFileNames.Add("MyCustomPage1.html");
```

```
app.UseFileServer(options);
```

10. Directory Browsing (Typically Disabled)

By default, ASP.NET Core **does NOT allow browsing directories**.
It must be explicitly enabled (usually not recommended in production).

11. Developer Exception Page

Default Production Behavior

When an unhandled exception occurs in production:

- Response = **500 Internal Server Error**
 - **No details are shown** for security
-

Enable Developer Exception Page (Development Only)

```
if (env.IsDevelopment())  
{  
    app.UseDeveloperExceptionPage();  
}
```



This shows:

- Stack trace
 - Error message
 - Request details
 - Route data
 - Cookies
 - Headers
-

Developer Exception Page Tabs

The UI displays diagnostic tabs:

1. Stack
2. Query
3. Cookies

- 4. Headers
- 5. Routing

Customizing Source Code Lines Displayed

```
if (env.IsDevelopment())
{
    var options = new DeveloperExceptionPageOptions
    {
        SourceCodeLineCount = 5
    };

    app.UseDeveloperExceptionPage(options);
}
```

This controls how many source code lines around the error are shown.

Correct Placement in Middleware Pipeline

Place **early**, before routing:

```
if (env.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
```

```
app.UseRouting();
```

Production Best Practice

Never enable it in production.

Instead use:

```
app.UseExceptionHandler("/Home/Error");
```

This redirects user to a custom error page.

12. Static Files & Default Files — Summary Table

Feature	Purpose
UseStaticFiles	Serves static resources
UseDefaultFiles	Rewrites request to default document
Default Files	index., <i>default.</i>
Custom Default	Can override defaults
UseFileServer	Combines static + default + directory browsing
Dev Exception Page	Debug only

13. Key Exam & Interview Points

- Static files live in **wwwroot**
 - Static Files Middleware must be enabled
 - Default file loading requires UseDefaultFiles()
 - Order matters in the middleware pipeline
 - Static files bypass authorization
 - Developer Exception Page is only for Development
 - Production must use UseExceptionHandler
 - Static file serving must occur **before routing**
- 

14. One-Line Memory Keys

- **wwwroot = public web root**
 - **UseStaticFiles() must be enabled**
 - **UseDefaultFiles() comes before UseStaticFiles()**
 - **Default files = index / default**
 - **Developer exception page = Development only**
-

1. What is Routing?

Routing is the **mechanism that maps incoming URLs to controllers and action methods**.

It determines:

Which controller?

Which action?

Which parameters?

Routing exists **inside the middleware pipeline**.

There are two main styles:

1. **Conventional Routing** (recommended for MVC apps)
2. **Attribute Routing** (recommended for APIs or custom URL patterns)

2. Enabling MVC Routing

`builder.Services.AddControllersWithViews();`

Then in the pipeline:

`app.UseRouting();`

3. Conventional Routing (Default)

Configured in Program.cs:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Meaning of Pattern

Segment	Description
controller	Controller name (without “Controller” suffix)
action	Action method
id?	Optional parameter

Examples

URL	Action Called
/	HomeController.Index()
/Home	HomeController.Index()
/Home/Index	HomeController.Index()
/Home/Details/1	HomeController.Details(1)

Optional Parameter Syntax

id? means the parameter is **optional**.

4. Attribute Routing

Attribute routing allows defining routes **directly on controllers and actions**.

Example

```
[Route("")]
[Route("Home")]
[Route("Home/Index")]
public IActionResult Index()
{
}
```



This supports:

- /
- /Home
- /Home/Index

Parameterized Route

```
[Route("Home/Details/{id}")]
public IActionResult Details(int id) { }
```

Optional Parameter in Attribute Routing

```
[Route("Home/Details/{id?}")]
public IActionResult Details(int? id) { }
```

5. Controller-Level Route Prefix

Define a prefix once per controller:

```
[Route("Home")]
public class HomeController : Controller
{
    [Route("")]
    [Route("Index")]
    public IActionResult Index()
    {
    }

    [Route("Details/{id?}")]
    public IActionResult Details(int? id)
    {
    }
}
```

Now supports URL patterns like:

- /Home
- /Home/Index
- /Home/Details
- /Home/Details/1

6. Route Tokens

Dynamic route placeholders:

Token	Meaning
[controller]	Controller class name minus “Controller”
[action]	Action method name

Example:

```
[Route("[controller]/[action]")]
public class HomeController : Controller
{
}
```

Advantage:

Routes survive renaming.

7. Conventional vs Attribute Routing — Comparison

Feature	Conventional Routing	Attribute Routing
Route Definition	In Program.cs	On controllers/actions
URL Style	Structured	Highly flexible
Best Use	MVC websites	APIs, REST services
Depends on names	Yes	No

8. Model Binding — Core Concept

Model Binding automatically maps HTTP request data to:

- Method parameters
- Model objects
- Properties

Value Sources

1. Form data
2. Route values
3. Query string
4. JSON body
5. Headers



9. Simple Example — Binding to Action Parameters

Request URL:

/home/details/2?name=Vita

Action:

```
public string Details(int? id, string name)
{
    return $"id = {id}, name = {name}";
}
```

Model Binder maps:

- 2 → id
- Vita → name

Requirement

Names must match.

10. Complex Model Binding

When action accepts a model object:

```
public RedirectToActionResult Create(Employee employee)
{
    var emp = _repository.Add(employee);

    return RedirectToAction("Details", new { id = emp.Id });
}
```

Binder maps:

- Form fields → Model properties

No manual parsing needed.

11. Model Binding Search Order

Order of lookup:

1. **Form fields**
 2. **Route values**
 3. **Query string**
-

12. HTTP GET vs POST — Action Selection

When controller has two methods with same name:

```
[HttpGet]
public ViewResult Create()
{
    return View();
}
```

```
[HttpPost]
public RedirectToActionResult Create(Employee employee)
{
    ...
}
```

Why Needed?

Otherwise, ASP.NET Core throws:

AmbiguousActionException

13. Developer Exception Page Reminder

Enable only in Development:

```
app.UseDeveloperExceptionPage();
```

Never use in Production.

Instead use:

```
app.UseExceptionHandler("/Home/Error");
```

14. 404 Error Handling (Two Types)

A. Business 404 — Record Not Found

```
public IActionResult Details(int id)
{
    var emp = repo.GetEmployee(id);

    if(emp == null)
    {
        Response.StatusCode = 404;
        return View("EmployeeNotFound", id);
    }

    return View(emp);
}
```



B. System 404 — Route Not Found

Middleware:

```
app.UseStatusCodePagesWithRedirects("/Error/{0}");
```

Controller:

```
[Route("Error/{statusCode}")]
public IActionResult HttpStatusCodeHandler(int statusCode)
{
    switch(statusCode)
    {
        case 404:
            ViewBag.ErrorMessage = "Resource not found";
            break;
    }

    return View("NotFound");
}
```

15. Default Route Pattern Breakdown

{controller=Home}/{action=Index}/{id?}

Element	Meaning
controller=Home	Default controller
action=Index	Default action
id?	Optional parameter

16. Practical Routing Rules to Remember

- Attribute routing overrides conventional routing
 - Routing is case-insensitive
 - Optional parameters use ?
 - Missing required parameter → 404
 - Route values flow into model binding
 - [controller] [action] tokens reduce refactoring risk
-

17. Key Interview & Exam Pointers

- MVC routing is defined in **Program.cs**
 - Two routing styles: Conventional & Attribute
 - Default route template always available
 - Model Binding auto-converts values to types
 - Model binding search order:
 1. Form fields
 2. Route values
 3. Query string
 - Missing or invalid enum → validation error unless nullable enum used
 - Developer Exception Page = Debug only
 - System 404 handled via middleware
 - Business 404 handled via controller logic
-

1. Why Do We Pass Data?

In MVC, a controller receives a request, performs logic, and must often **send data to the View for rendering**.

ASP.NET Core MVC provides **four main techniques**:

1. **ViewData**
 2. **ViewBag**
 3. **TempData**
 4. **Strongly Typed ViewModels (Best Practice)**
-

2. ViewData

Purpose

Pass data **from Controller → View**

Valid **only for the current request**

Type

ViewDataDictionary — uses **string keys**

Syntax

Controller

```
ViewData["Message"] = "Hello";
```

View

```
@ViewData["Message"]
```

Key Characteristics

Feature	Detail
Lifetime	Current request only
Data structure	Key-value dictionary
Casting	Required
Compile-time safety	None
Null handling	Must be checked
Introduced	MVC 1.0

3. ViewBag

Purpose

Also sends data **Controller** → **View**, same lifetime as ViewData.

Type

dynamic wrapper over ViewData.

Syntax

Controller

```
ViewBag.Message = "Hello";
```

View

```
@ViewBag.Message
```

Key Characteristics

Feature	Detail
Lifetime	Current request
Data structure	Dynamic object
Casting	Not required
Compile-time safety	None
Internally stored in	ViewData
Introduced in	MVC 3.0



4. ViewData vs ViewBag — Quick Comparison

Feature	ViewData	ViewBag
Type	Dictionary	Dynamic wrapper
Access style	ViewData["x"]	ViewBag.x
Casting required	Yes	No
Lifetime	Current request	Current request
Redirect survives	No	No
Backend storage	Dictionary	ViewData

Important:

Both lose data after redirect.

5. TempData

Purpose

Used to pass data:

- Controller → View
- Controller → Controller
- Action → Action

And **persists across redirect**.

Typical uses:

- Success messages
- Error notifications
- Form submission messages
- Redirect flows

Syntax

Store

```
TempData["Message"] = "Saved Successfully";  
return RedirectToAction("Next");
```

Read

```
var msg = TempData["Message"];
```

Internal Storage

Session-based dictionary.

Lifetime Behavior

Action	Result
Read once	Removed automatically
Keep()	Retains value
Peek()	Reads without removing

Methods

TempData.Keep();

TempData.Keep("key");

TempData.Peek("key");

6. Summary Table — ViewData vs ViewBag vs TempData

Feature	ViewData	ViewBag	TempData
Type	Dictionary	Dynamic wrapper	Session Dictionary
Lifetime	Current request	Current request	Current + next request
Redirect support	No	No	Yes
Casting	Required	No	Required
Storage	Memory	ViewData	Session

7. Best Practice — Use Strongly Typed ViewModels

Why?

- Compile-time safety
- IntelliSense
- Cleaner code
- No runtime errors
- Structured data passing



Example — ViewModel Class

```
public class HomeDetailsViewModel
{
    public Employee Employee { get; set; }
    public string PageTitle { get; set; }
}
```

Controller

```
public ActionResult Details()
{
    var viewModel = new HomeDetailsViewModel
    {
        Employee = _employeeRepository.GetEmployee(1),
        PageTitle = "Employee Details"
    };

    return View(viewModel);
}
```

View

@model EmployeeManagement.ViewModels.HomeDetailsViewModel

<h3>@Model.PageTitle</h3>

<div>@Model.Employee.Name</div>

8. Enum Binding Issue — Important Note

Enums **cannot store empty string**, so drop-downs cause validation errors.

Correct Solution — Use Nullable Enum

[Required]

```
public Dept? Department { get; set; }
```

View

<select asp-for="Department"

asp-items="Html.GetEnumSelectList<Dept>()">

<option value="">Please Select</option>

</select>

9. TempData vs ModelState

Feature	TempData	ModelState
Purpose	Carry data across redirects	Store validation errors
Lifetime	1-2 requests	Current request only

10. Business 404 Handling Reminder

Example:

```
public ActionResult Details(int id)
{
    var emp = repo.GetEmployee(id);

    if(emp == null)
    {
        Response.StatusCode = 404;
        return View("EmployeeNotFound", id);
    }

    return View(emp);
}
```

11. System 404 Handling via Middleware

```
app.UseStatusCodePagesWithRedirects("/Error/{0}");
```

Controller:

```
[Route("Error/{statusCode}")]
```

```
public IActionResult HttpStatusCodeResult(int statusCode)
```

```
{
    switch(statusCode)
    {
        case 404:
            ViewBag.ErrorMessage = "Resource not found";
            break;
    }

    return View("NotFound");
}
```

1. Model Binding — Core Concept

Model Binding is the process that **automatically maps incoming HTTP request data to controller action parameters or model objects**.

Data Sources Used

Model Binder searches values in this order:

1. **Form fields**
2. **Route values**
3. **Query string**
4. JSON body
5. Headers

You **do not need to manually read Request objects**.

2. Simple Model Binding Example

Request URL:

/home/details/2?name=Vita

Action:

```
public string Details(int? id, string name)
{
    return $"id = {id}, name = {name}";
}
```

Mapping done automatically:

- id = 2
- name = Vita

Requirement:

Parameter names must match input names.

3. Complex Model Binding (Binding to Model Object)

When a form posts data matching a model:

```
public RedirectToActionResult Create(Employee employee)
{
    var emp = _repository.Add(employee);

    return RedirectToAction("Details", new { id = emp.Id });
}
```

Model Binder automatically maps:

- Each form field → Corresponding model property

No manual parsing is required.

4. HTTP GET vs POST — Action Selection

When the same action name handles different HTTP verbs, you must explicitly specify method attributes.

GET — Render Form

[HttpGet]

```
public ViewResult Create()
{
    return View();
}
```

POST — Submit Form

[HttpPost]

```
public RedirectToActionResult Create(Employee employee)
{
    ...
}
```



Why Required?

If both actions have the same name and no attributes, MVC throws:

AmbiguousActionException

Because runtime cannot determine which to execute.

5. Validation in ASP.NET Core MVC

Validation ensures that user input meets defined requirements **before processing**.

Validation Attributes — Applied to Model Properties

Example:

```
public class Employee
{
    public int Id { get; set; }

    [Required]
    [MaxLength(50)]
```

```
public string Name { get; set; }
```

```
[Required]
```

```
[EmailAddress]
```

```
public string Email { get; set; }
```

```
public Dept Department { get; set; }
```

```
}
```

6. Common Validation Attributes — With Purpose

Attribute	Purpose
[Required]	Value must be entered
[Range]	Validates numeric limits
[MinLength]	Minimum string length
[MaxLength]	Maximum string length
[Compare]	Compare two fields
[RegularExpression]	Pattern validation
[EmailAddress]	Email format validation

7. Checking Validation Result

Validation occurs automatically during action execution.

You must check:

```
if (ModelState.IsValid)
```

```
{
```

```
...
```

```
}
```

If Invalid

Return the same view so UI displays errors.

8. Displaying Errors in the View

Place validation message elements in Razor view.

Field-Level Error Display

```
<span asp-validation-for="Name"></span>
```

Summary Error Display

```
<div asp-validation-summary="All"></div>
```

9. Handling Enum Drop-Down Validation (Important Issue)

Enums **cannot store empty string**.

So when a dropdown includes “Select” option, validation may fail.

Correct Fix — Use Nullable Enum

[Required]

```
public Dept? Department { get; set; }
```

View Example

```
<select asp-for="Department"
        asp-items="Html.GetEnumSelectList<Dept>()">
    <option value="">Please Select</option>
</select>
```

This prevents invalid value exceptions.

10. TempData vs ModelState — Key Difference

Feature	TempData	ModelState
Purpose	Share data across redirect	Stores validation errors
Lifetime	Current + next request	Current request only

11. Business 404 vs System 404 Reminder

Business 404 (Record not found)

```
public ActionResult Details(int id)
{
    var emp = repo.GetEmployee(id);

    if(emp == null)
    {
        Response.StatusCode = 404;
        return View("EmployeeNotFound", id);
    }

    return View(emp);
}
```

System 404 (Route not found)

Middleware:

```
app.UseStatusCodePagesWithRedirects("/Error/{0}");
```

Controller:

```
[Route("Error/{statusCode}")]
```

```
public IActionResult HttpStatusCodeHandler(int statusCode)
```

```
{  
    switch(statusCode)  
    {  
        case 404:  
            ViewBag.ErrorMessage = "Resource not found";  
            break;  
    }  
}
```

```
    return View("NotFound");  
}
```

12. Developer Exception Page (Debug Only)

Enable in Development:

```
app.UseDeveloperExceptionPage();
```

Never in Production.

Production alternative:

```
app.UseExceptionHandler("/Home/Error");
```



13. Model Binding — Key Rules

- Parameter names must match input names
 - Search order:
 1. Form
 2. Route
 3. Query
 - Nullable parameter supports missing value
 - Invalid value causes validation error
 - Enum must be nullable if empty value allowed
-

1. What is Middleware?

Middleware is a **software component that processes HTTP requests and responses** in the ASP.NET Core pipeline.

Every request:

- Enters the pipeline at the top
- Passes through each middleware component in order
- Reaches terminal middleware (e.g., MVC endpoint)
- Response travels back up in reverse order

Key Rule

Middleware executes in the exact order registered in Program.cs.

Order affects:

- Security
- Routing
- Sessions
- Authentication
- Performance
- Error handling



2. Request Flow Direction

Direction	Meaning
Request →	Top → Bottom through pipeline
Response ←	Bottom → Top back through pipeline

3. Common Middleware Examples

- Static Files
- HTTPS redirection
- Exception handling
- Authentication / Authorization
- Routing
- MVC execution

- Logging
 - Custom middleware
-

4. Request Delegates

A **request delegate** is a function that can process requests.

Delegates are created using:

- Use()
- Run()
- Map()

Delegates may:

- Pass execution to the next delegate
- Or terminate the pipeline

They may be:

- Inline (lambda)
 - Implemented as middleware classes
-

5. Difference Between Use() and Run()

Use() — Non-terminal Middleware

It may continue the pipeline using next()

```
app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("Before");
    await next(); // Call next component
    await context.Response.WriteAsync("After");
});
```

Run() — Terminal Middleware

Does **NOT** call the next delegate. Ends pipeline.

```
app.Run(async context =>
{
    await context.Response.WriteAsync("Terminal Middleware");
});
```

If there are multiple Run() calls, **only the first one executes**.

6. Execution Flow Example

```
app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("M1 IN\n");
    await next();
    await context.Response.WriteAsync("M1 OUT\n");
});
```

```
app.Use(async (context, next) =>
{
    await context.Response.WriteAsync("M2 IN\n");
    await next();
    await context.Response.WriteAsync("M2 OUT\n");
});
```

```
app.Run(async context =>
{
    await context.Response.WriteAsync("M3 Handling Request\n");
});
```

Output

M1 IN

M2 IN

M3 Handling Request

M2 OUT

M1 OUT



Explanation

- Request moves downward through Use() middleware
- Run() handles the request
- Response travels back upward executing post-logic

7. Built-In Middleware — Recommended Order

Correct pipeline structure:

UseExceptionHandler

UseHttpsRedirection

UseStaticFiles

UseRouting

UseAuthentication

UseAuthorization

UseEndpoints / MapControllerRoute

Why This Order Matters

Middleware	Reason
Error handling first	Catch all downstream exceptions
HTTPS redirection	Force security early
Static files early	Serve without running MVC
Routing before authentication?	No — authentication applies after routing
Authentication before authorization	Identity before access check
Endpoint execution last	Executes the MVC pipeline

8. Static File Middleware Position

Static files should come **before** routing:

```
app.UseStaticFiles();
```

```
app.UseRouting();
```

Because:

- Static files bypass MVC
 - Returning them early improves performance
-

9. HTTPS Middleware

UseHttpsRedirection

Redirects HTTP → HTTPS.

UseHsts

Adds Strict-Transport-Security header.

(Production only.)

10. Exception Handling Middleware

Production

```
app.UseExceptionHandler("/Home/Error");
```

Development

```
app.UseDeveloperExceptionPage();
```

11. MVC & Routing Middleware

Enable routing:

```
app.UseRouting();
```

Define endpoints:

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Controllers are **transient** — **created per request**.

12. Custom Middleware — Two Ways to Create

A. Implementing IMiddleware

```
public class CustomMiddlewareDemo : IMiddleware  
{  
    public async Task InvokeAsync(HttpContext context, RequestDelegate next)  
    {  
        await context.Response.WriteAsync("Hello from Custom Middleware");  
        await next(context);  
    }  
}
```



Register in DI

```
builder.Services.AddTransient<CustomMiddlewareDemo>();
```

Add to Pipeline

```
app.UseMiddleware<CustomMiddlewareDemo>();
```

B. Middleware Class + Extension Method

Middleware Class

```
public class Middleware  
{  
    private readonly RequestDelegate _next;  
  
    public Middleware(RequestDelegate next) => _next = next;
```

```

public async Task Invoke(HttpContext context)
{
    await context.Response.WriteAsync("Hello Custom Middleware\n");
    await _next(context);
}
}

```

Extension Method

```

public static class MiddlewareExtensions
{
    public static IApplicationBuilder UseMiddleware(this IApplicationBuilder app)
    => app.UseMiddleware<Middleware>();
}

```

Use in Program.cs

```
app.UseMiddleware();
```

13. Request Delegates Summary Table

Command	Pipeline Behavior
Use()	May continue to next delegate
Run()	Ends pipeline
Map()	Branches pipeline paths

14. Response Flow Concept

Very important for interviews

Request enters pipeline

↓

Each middleware optionally runs pre-logic

↓

Terminal middleware sends response

↑

Each middleware runs post-logic during return

15. Key Security & Performance Notes

- Wrong middleware order can break application logic
 - Static files are public by default
 - Authentication must run **before authorization**
 - Response caching should run after static files
 - Exception handling should be first in pipeline
 - Terminal middleware prevents further middleware execution
-

16. One-Page Revision Points

- Middleware executes sequentially in registration order
 - Use() = chainable
 - Run() = terminal delegate
 - Map() = branch routing
 - Response flows back upward
 - Built-in middleware has recommended order
 - Static files must be served before MVC routing
 - Custom middleware can be:
 - IMiddleware class
 - Extension-based
 - HTTPS redirection and HSTS increase security
 - Developer Exception Page → Development only
-

1. Why State Management Is Needed

HTTP is **stateless** — meaning each request is independent.

State management enables storing **user-specific data across requests**, such as:

- Login identity
- Shopping cart
- Preferences
- Form progress

ASP.NET Core supports several storage mechanisms.

2. State Storage Options in ASP.NET Core

Approach	Storage	Scope
Cookies	Client browser	Client-side
Session	Server memory / Distributed store	Per-user
TempData	Session / Cookies internally	One request + redirect
Query Strings	URL	Client-side
Hidden Fields	Form fields	Client-side
HttpContext.Items	Server memory	Per-request
Cache	Server memory	App-wide

Part A — Cookies in ASP.NET Core MVC

3. What Is a Cookie?

A **cookie** is a small **key-value storage saved in the client browser**.

Created by server → stored on client → sent back with every request.

Use Cases

- Remember-Me login
- Cart items
- User preferences
- Tracking / personalization
- Authentication metadata

4. Types of Cookies

1. Persistent Cookies

- Stored on disk
- Remain after browser closes
- Require explicit expiry

```
options.Expires = DateTime.Now.AddDays(7);
```

2. Session (Non-Persistent) Cookies

- No expiry set
- Stored in memory
- Deleted when browser closes

```
Response.Cookies.Append("key", "value");
```

5. Key Cookie Characteristics

Cookies are:

- Stored client-side
- Browser-specific
- User-specific
- Max size ~ **4KB per cookie**
- Automatically sent in every request

Chrome cookie ≠ Firefox cookie (not shared).

6. Creating (Writing) a Cookie

```
public IActionResult Create()
{
    string key = "DemoCookie";
    string value = DateTime.Now.ToString();

    CookieOptions options = new CookieOptions();
    options.Expires = DateTime.Now.AddDays(7);

    Response.Cookies.Append(key, value, options);

    return View();
}
```

7. Reading a Cookie

```
public IActionResult Read()
{
    string key = "DemoCookie";

    var value = Request.Cookies[key];

    return View();
}
```

8. Deleting a Cookie

```
Response.Cookies.Delete("DemoCookie");
```

Or force expiry:

```
Response.Cookies.Append("DemoCookie", "",
    new CookieOptions { Expires = DateTime.Now.AddDays(-1) });
```

9. CookieOptions — Very Important

```
var options = new CookieOptions()
{
    Domain = "localhost",
    Path = "/",
    Expires = DateTime.Now.AddDays(7),
    Secure = true,
    HttpOnly = true,
    IsEssential = true
};
```

Property	Meaning
Domain	Scope to domain
Path	URL scope
Expires	Expiry date
Secure	HTTPS-only
HttpOnly	Prevent JavaScript access
IsEssential	Allowed under GDPR settings

10. “Remember Me” Concept

When selected:

- Username or ID stored in cookie
- Expiry manually defined

```
if(model.Remember)
{
    Response.Cookies.Append("Ukey", model.Email,
        new CookieOptions(){ Expires = DateTime.Now.AddDays(7) });
}
```

Retrieve later:

```
Request.Cookies["Ukey"];
```

11. Reading Cookies in Views

Register accessor:

```
builder.Services.AddSingleton<IHttpContextAccessor, HttpContextAccessor>();
```

View:

```
@inject IHttpContextAccessor HttpContextAccessor
```

```
@HttpContextAccessor.HttpContext.Request.Cookies["UserName"]
```

12. Cookie Use Cases

- Authentication
 - Remember-Me login
 - User preferences
 - Shopping carts
 - Tracking
 - Multi-page forms
-

13. Cookie Limitations & Risks

Issue	Description
Size limit	~4KB
Security	Vulnerable to XSS/CSRF
Editable	User may delete / modify
Browser dependent	Storage behavior varies
Overhead	Sent on every request

Never store sensitive data in plain text cookies.

14. Cookie Security Best Practices

- Set HttpOnly = true
 - Set Secure = true in production
 - Avoid storing confidential data
 - Encrypt if required
-

15. One-Line Cookie Memory Notes

- Persistent cookie → survives browser close
 - Session cookie → removed on close
 - HttpOnly → blocks JavaScript
 - Secure → HTTPS-only
-

Part B — Session State in ASP.NET Core MVC

16. What Is Session State?

Session stores **user-specific data on the server across multiple requests.**

Typical Use Cases

- Login state
 - Shopping cart
 - Multi-step forms
 - Temporary user preferences
-

17. How Session Works Internally

1. First request received
2. Server creates **Session ID**
3. Session ID stored in **session cookie**
4. Server stores data using Session ID
5. Browser sends cookie in later requests
6. Session expires after idle timeout

Sessions are **user-specific and browser-specific**.

18. Session Characteristics

- Stored on server
 - Default mode = **In-Memory (In-Proc)**
 - Default timeout = **20 minutes**
 - New session created after expiration
 - Exists only once data is stored
-

19. Enabling Session in ASP.NET Core

Step 1 — Register Session Services

```
builder.Services.AddDistributedMemoryCache();
```

```
builder.Services.AddSession(options =>
{
    options.IdleTimeout = TimeSpan.FromMinutes(20);
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
});
```

Step 2 — Enable Session Middleware

```
app.UseSession();
```

20. Store Data in Session

```
HttpContext.Session.SetString("_Name", "DAC");
```

```
HttpContext.Session.SetInt32("_Age", 24);
```

21. Read Data from Session

```
ViewBag.Name = HttpContext.Session.GetString("_Name");
```

```
ViewBag.Age = HttpContext.Session.GetInt32("_Age");
```

Always check null when retrieving.

22. Remove Session Data

```
HttpContext.Session.Remove("_Name");
```

```
HttpContext.Session.Clear();
```

23. Accessing Session in Views

```
@using Microsoft.AspNetCore.Http
```

```
<h1>@Context.Session.GetString("_Name")</h1>
```

or via DI:

```
@inject IHttpContextAccessor HttpContextAccessor
```

```
@HttpContextAccessor.HttpContext.Session.GetString("_UserName")
```

24. Session Timeout Options

Property	Meaning
IdleTimeout	Max inactive duration
IOTimeout	Max read/write timeout
Cookie.Name	Session cookie name
HttpOnly	Prevent JavaScript access
IsEssential	Works without consent

Example:

```
options.IdleTimeout = TimeSpan.FromMinutes(30);
```

25. Session Storage Types

1. In-Memory (In-Proc)

- ✓ Fast
 - ✓ Easiest
 - ✗ Lost on restart
 - ✗ Not for load-balanced environments
-

2. Distributed Session (SQL / Redis)

- ✓ Survives restart
- ✓ Supports web-farm clustering
- ✗ Slightly slower
- ✗ Requires setup

Example SQL configuration:

```
builder.Services.AddDistributedSqlServerCache(options =>
{
    options.ConnectionString = builder.Configuration.GetConnectionString("EFCoreDBConnection");
    options.SchemaName = "dbo";
    options.TableName = "MySessions";
});
```

26. Session vs Cookies — Comparison

Feature	Session	Cookies
Storage	Server	Browser
Security	More secure	Less secure
Size Limit	Large (memory)	~4KB
Lifetime	Configurable	Configurable
Best For	Sensitive / larger data	Small preference data

27. When To Use Session

Use session when data is:

- Temporary
- User-specific
- Security-sensitive

- Must not be stored on client

Examples:

- Login state
 - Shopping cart
 - Multi-page forms
-

28. Session Limitations

- Requires server memory
 - Session can expire unexpectedly
 - Not shared across browsers
 - Not permanent storage
-

29. One-Line Session Memory Keys

- Session ID stored in cookie
- Session data stored on **server**
- Default timeout = **20 minutes**
- Must enable Session in Program.cs
- Use for temporary per-user data



Part C — TempData (Quick Recap)

TempData:

- Persists **one request + redirect**
- Stored via session internally
- Used for:
 - Success messages
 - Redirect flows

Supports:

`TempData.Keep();`

`TempData.Peek();`

Part D — Security & Best Practices Recap

- Cookies → client-side
 - Session → server-side
 - Avoid storing sensitive data in cookies
 - Encrypt when needed
 - Use HttpOnly + Secure flags
 - Session must be explicitly enabled
 - Distributed session for load-balanced apps
-



1. What is Caching?

Caching is the process of **storing frequently used data so that future requests can be served faster.**

Core Terms

- **Cache Hit** → Data found in cache
- **Cache Miss** → Data not found

Benefits of Caching

- ✓ Improves performance
- ✓ Reduces response time
- ✓ Reduces server load and database calls

Rule

Only cache data that **does not change frequently.**

2. Types of Caching in ASP.NET Core

ASP.NET Core supports multiple caching strategies:

1. **In-Memory Caching (Server RAM)**
 2. **Distributed Caching (SQL / Redis etc.)**
 3. **Response Caching (Browser / Proxy Cache)**
 4. **View-Level Caching (Cache Tag Helper)**
-

Part A — In-Memory Caching (Server-Side Cache)

3. Characteristics

Stored in **server RAM.**

- ✓ Extremely fast
- ✓ Easy to implement
- ✗ Lost when server restarts
- ✗ Not shared across multiple servers

Best for **small-to-medium apps on single server.**

4. Enabling In-Memory Cache

```
builder.Services.AddMemoryCache();
```

5. Inject IMemoryCache

```
public class HomeController : Controller
{
    private readonly IMemoryCache _cache;

    public HomeController(IMemoryCache cache)
    {
        _cache = cache;
    }
}
```

6. Store Cache Item

```
_cache.Set("CacheTime", DateTime.Now);
```

7. Read Cache Item

```
var time = _cache.Get<DateTime>("CacheTime");
```

8. Recommended Pattern — Check Before Creating

```
DateTime cacheTime;

if(!_cache.TryGetValue("CacheTime", out cacheTime))
{
    cacheTime = DateTime.Now;

    var options = new MemoryCacheEntryOptions()
        .SetSlidingExpiration(TimeSpan.FromSeconds(30));

    _cache.Set("CacheTime", cacheTime, options);
}
```

9. Cache Expiration Types

1. Sliding Expiration

Timer resets on each access.

```
.SetSlidingExpiration(TimeSpan.FromSeconds(30));
```

2. Absolute Expiration

Expires at fixed duration or timestamp.

```
_cache.Set("key", value, TimeSpan.FromMinutes(10));
```

Or

```
_cache.Set("key", value, new DateTime(2026,1,1));
```

10. Remove Cached Item

```
_cache.Remove("CacheTime");
```

11. Useful Cache Methods

Method	Purpose
Set()	Store value
Get()	Retrieve value
TryGetValue()	Safely retrieve
GetOrCreate()	Create if missing
Remove()	Delete entry

Example

```
_cache.GetOrCreate("CacheTime", entry =>
{
    entry.SlidingExpiration = TimeSpan.FromSeconds(45);
    return DateTime.Now;
});
```

Part B — View-Level Caching (Cache Tag Helper)

12. Cache Only A Section of Razor View

```
<cache>
    <p>@DateTime.Now</p>
</cache>
```

13. Cache Tag Helper Attributes

Attribute	Meaning
enabled	Turn cache on/off
expires-on	Expire at specific date
expires-after	Expire after duration
expires-sliding	Sliding expiration
vary-by-header	Cache varies per header
vary-by-query	Cache varies per query string
vary-by-route	Cache varies per route

Examples

Absolute Expiry

```
<cache expires-on="@new DateTime(2025,1,29,17,02,0)">
```

Sliding Expiry

```
<cache expires-sliding="@TimeSpan.FromSeconds(60)">
```

Part C — Response Caching

14. What is Response Caching?

Stores **entire HTTP responses** in:

- Browser cache
- Proxy caches

15. Enable Response Caching

```
builder.Services.AddResponseCaching();
```

```
app.UseResponseCaching();
```

16. Apply to Action Method

```
[ResponseCache(Duration = 3600)]
```

```
public IActionResult Index() { }
```

Meaning → cache response for **1 hour**.

17. Disable Caching for Action

[ResponseCache(Location = ResponseCacheLocation.None, NoStore = true)]

Used for dynamic / sensitive data.

Part D — Distributed Caching (Concept)

18. When Distributed Cache Is Needed

Used when:

- ✓ Multiple servers (load balanced)
- ✓ Cloud hosting
- ✓ Need persistence across restarts

Examples:

- SQL Server Cache
- Redis Cache

Distributed cache is **shared across servers**.

19. In-Memory Cache vs Distributed Cache

Feature	In-Memory	Distributed
Storage	Server RAM	External store
Speed	Very fast	Fast
Multi-Server Support	No	Yes
Survives Restart	No	Yes

Part E — Service Lifetime Summary (Revision Topic)

20. Service Lifetimes (Related to Caching & DbContext)

Lifetime Instance Created

Transient Per request

Scoped Per HTTP request

Singleton Once per app

21. AddDbContext vs AddDbContextPool

Method	Behavior
AddDbContext	New DbContext per request
AddDbContextPool	Reuses DbContext instances

Use Pooling **ONLY IF**

DbContext does **not** maintain state.

Part F — Important Interview Points

22. High-Value Q&A Concepts

- Cache **only stable data**
- Cache reduces DB calls
- Session $\neq$ Cache

Concept Purpose

Session Per-user data storage

Cache Application-wide reusable data

- Response caching relies partly on browser
 - In-memory cache clears on restart
 - Sliding vs Absolute expiration differences
-



1. Views in ASP.NET Core MVC — Purpose

Views are responsible for **rendering UI to the user** using Razor syntax.

They:

- Display data received from Controllers
- Use Models or ViewModels
- Support Tag Helpers & HTML Helpers
- Integrate with layout pages

Views should **not contain business logic** — only presentation logic.

Part A — ViewModels (Recap & Deep Understanding)

2. Why Use ViewModels?

A ViewModel is used when:

- A View requires **multiple model objects**
- A View needs **additional UI data** not stored in the database
- You want **strong typing + IntelliSense**
- You want **compile-time safety**



Benefits

- ✓ Structured data transfer
 - ✓ Cleaner separation of concerns
 - ✓ Avoids ViewData/ViewBag reliance
 - ✓ Prevents runtime errors
-

3. ViewModel Example

Class

```
public class HomeDetailsViewModel  
{  
    public Employee Employee { get; set; }  
    public string PageTitle { get; set; }  
}
```

Controller

```
public ActionResult Details()
{
    var viewModel = new HomeDetailsViewModel
    {
        Employee = _employeeRepository.GetEmployee(1),
        PageTitle = "Employee Details"
    };

    return View(viewModel);
}
```

View

```
@model EmployeeManagement.ViewModels.HomeDetailsViewModel
```

```
<h3>@Model.PageTitle</h3>
```

```
<div>@Model.Employee.Name</div>
```

Part B — Layout Pages & Sections

4. Layout Pages — Purpose

Layout pages act like **master templates** to maintain consistent UI across all pages.

They contain shared elements such as:

- Header
- Footer
- Navigation bar
- Scripts

Similar to **Master Pages in ASP.NET MVC 5**.

5. Sections

Sections allow **individual Views to insert content into defined regions inside the Layout.**

Define Section Placeholder in Layout

```
@RenderSection("Scripts", required: false)
```

- required: false → Section optional
- required: true → View must define it

Define Section in View

```
@section Scripts {  
    <script src="~/js/custom.js"></script>  
}
```

6. Summary — Layout vs Section

Feature	Layout	Section
Purpose	Common template	Insert view-specific content
Contains RenderBody()	Yes	No
Optional	No	Yes / No
Defined in	Layout page	View

Part C — _ViewImports.cshtml

7. Purpose of _ViewImports.cshtml

This file **centralizes imports and directives** shared by all Razor views to avoid repetition.

Typical contents include:

```
@using EmployeeManagement.Models
```

```
@using EmployeeManagement.ViewModels
```

```
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

8. Hierarchy & Precedence

View lookup order:

1. View file
2. _ViewImports in current folder
3. Root _ViewImports

Meaning:

Local _ViewImports overrides higher level.

Part D — Tag Helpers in ASP.NET Core MVC

9. What Are Tag Helpers?

Tag Helpers **extend standard HTML tags with server-side functionality** while keeping HTML clean and readable.

They:

- Are strongly-typed
- Support IntelliSense
- Perform routing integration
- Auto-generate form links & routes

They **replace HTML Helpers in many scenarios.**

10. Anchor Tag Helper

Example

```
<a asp-controller="home" asp-action="details" asp-route-id="5">
```

```
View
```

```
</a>
```

This automatically renders a valid URL based on routing.

11. Image Tag Helper (Cache Busting)

```

```

asp-append-version="true"

Appends a hash to file path to avoid stale browser cache.

12. Form Tag Helper

```
<form asp-action="create" method="post">
```

Generates form post URL automatically.

13. Input Tag Helper

```
<input asp-for="Name" />
```

Automatically binds to model property.

Part E — Environment Tag Helper

14. Purpose

Allows loading **different resources based on environment**, such as Development vs Production.

Development Environment Example

```
<environment include="Development">  
  <link rel="stylesheet" href="~/lib/bootstrap.css" />  
</environment>
```

CDN With Fallback Example

```
<environment exclude="Development">  
  <link rel="stylesheet"  
    href="https://cdn..."  
    asp-fallback-href="~/css/bootstrap.css" />  
</environment>
```

This ensures site works even if CDN fails.

15. What Is a Partial View?

A **partial view** is a **reusable UI fragment** that can be rendered inside:

- Views
- Layout pages

Used for:

- Headers
- Navigation bars
- Footers
- Widgets
- Login panels
- Cards

16. Where To Place Partial Views

Usage	Location
Used by multiple controllers	/Views/Shared
Used by single controller	That controller's Views folder

Naming typically begins with `_`, e.g.

`_HeaderNavBar.cshtml`

17. Example Partial View

```
<header>
```

```
<nav class="navbar navbar-light bg-white border-bottom box-shadow mb-3">
```

```
<div class="container-fluid">
```

```
<a class="navbar-brand" asp-controller="Home" asp-action="Index">
```

```
MVC_Core
```

```
</a>
```

```
</div>
```

```
</nav>
```

```
</header>
```

18. Rendering Partial Views (3 Techniques)

1. <partial> Tag Helper — Recommended

```
<partial name="_HeaderNavBar" />
```

2. Html.PartialAsync() — Returns Content

```
@await Html.PartialAsync("_HeaderNavBar")
```

3. Html.RenderPartialAsync() — Streams Output

```
@{  
    await Html.RenderPartialAsync("_HeaderNavBar");  
}
```

Difference

Method	Returns?	Performance
PartialAsync()	HTML content	Normal
RenderPartialAsync()	Writes directly to response	Faster

19. Passing Model to Partial View (Optional)

```
@model Employee
```

```
<div>@Model.Name</div>
```

20. Partial View vs Layout View — Comparison

Feature	Partial View	Layout View
Purpose	Reusable fragment	Page template
Contains RenderBody()	No	Yes
Used inside layout	Yes	N/A
Reusable across pages	Yes	Yes

21. When To Use Partial Views

Use for:

- Header
- Footer
- Menu
- Login panel
- Reusable list/table/card

Avoid for **complex business logic** — use **View Components** instead.

Part G — Developer Exception Page (UI Tabs Reminder)

For quick recollection:

Tabs include:

1. Stack
2. Query
3. Cookies
4. Headers
5. Routing



1. End-to-End MVC Request Lifecycle (Complete Flow)

This section shows **how everything you learned fits together**.

Step-by-Step Flow

1. Browser Sends Request

- Request enters server

2. Middleware Pipeline Begins

- Order matters
- Typical order:
 - UseExceptionHandler / DeveloperExceptionPage
 - UseHttpsRedirection
 - UseStaticFiles
 - UseRouting
 - UseAuthentication
 - UseAuthorization
 - MapControllerRoute

3. Static File Check

- If request matches a file in **wwwroot**
- Static file is returned directly
- MVC is **never invoked**

4. Routing Resolves Controller & Action

- Conventional routing:
 - {controller=Home}/{action=Index}/{id?}
- Attribute routing if applied

5. Controller Instance Is Created

- Controllers are **Transient** (new instance per request)

6. Dependency Injection Injects Services

- Services registered as:
 - Singleton
 - Scoped
 - Transient



7. Model Binding Maps Input Data

From:

- Form fields
- Route values
- Query strings
- JSON
- Headers

Search Order

Form → Route → Query

8. Validation Runs Automatically

Using attributes like:

- [Required]
- [MaxLength]
- [Range]
- [EmailAddress]

Checked via:

`if(ModelState.IsValid)`

9. Action Executes Business Logic

- Calls repositories
- Uses DbContext (Scoped lifetime)

10. Data Passed to View

Using:

- ViewModel (recommended)
- ViewData
- ViewBag
- TempData (for redirects)

11. Razor View Renders UI

- Layout applies template
- Tag Helpers resolve links & binding
- Sections inject page-specific scripts

12. Response Sent Back

- Response flows back up middleware pipeline
- Logging / Post-processing happens now



2. Error Handling — Complete Picture

There are **two categories of errors**:

A. Business / Application Errors (Record Not Found)

Handled in controller:

```
if(emp == null)
{
    Response.StatusCode = 404;

    return View("EmployeeNotFound", id);
}
```

B. System Errors (Route / HTTP Code Errors)

Handled via middleware:

```
app.UseStatusCodePagesWithRedirects("/Error/{0}");
Controller:
[Route("Error/{statusCode}")]
public IActionResult HttpStatusCodeHandler(int statusCode)
{
    switch(statusCode)
    {
        case 404:
            ViewBag.ErrorMessage = "Resource not found";
            break;
    }

    return View("NotFound");
}
```

3. Developer Exception Page vs Production Error Handling

Environment	Middleware
Development	UseDeveloperExceptionPage()
Production	UseExceptionHandler("/Home/Error")

Developer Exception Page Tabs Include

1. Stack
2. Query
3. Cookies
4. Headers
5. Routing

4. Validation — Full Flow

1. Request posted
2. Model Binder maps data
3. Validation attributes run
4. Invalid → ModelState contains errors
5. Action checks:

if (!ModelState.IsValid)

6. Errors displayed via:

<div asp-validation-summary="All"></div>

Enum Validation Rule (Very Important)

Enums **must be nullable** when “Select” option exists

[Required]

```
public Dept? Department { get; set; }
```

5. State Management — Where to Use What

Mechanism	Storage	Lifetime	Best For
ViewData	Memory	Current request	Simple data
ViewBag	Memory	Current request	Simple data
TempData	Session	Redirects	Notifications
Session	Server	Configurable	Login & cart
Cookies	Browser	Configurable	Preferences
Cache	Server	Depends	Performance

6. Static Files — Operational Rules

- Served from **wwwroot**
- Require `UseStaticFiles()`
- Must be before routing
- Bypass MVC & security
- Default files require:

```
app.UseDefaultFiles();
```

```
app.UseStaticFiles();
```

Default File Order

1. `index.htm`
 2. `index.html`
 3. `default.htm`
 4. `default.html`
-

7. Middleware — Key Behavioral Rules

- Order defines behavior
 - `Use()` continues pipeline
 - `Run()` terminates pipeline
 - Response returns upward
 - Custom middleware can:
 - Use `IMiddleware`
 - Or extension-based class
-

8. Dependency Injection — Lifetime Rules

Lifetime	Instance
Singleton	Once per application
Scoped	Once per HTTP request
Transient	Every request

Recommended

- `DbContext` → Scoped

- Business Services → Scoped / Transient
 - Configuration → Singleton
-

9. Configuration — Override Priority

Highest → Lowest:

1. Command-line arguments
2. Environment variables
3. User Secrets
4. appsettings.{Environment}.json
5. appsettings.json

Command-line always wins.

10. Routing — Full Understanding

Default Conventional Route

{controller=Home}/{action=Index}/{id?}

Attribute Routing

[Route("Home/Details/{id?}")]

Tokens

- [controller]
 - [action]
-



11. Caching — Revision Summary

Types

- ✓ In-Memory Cache
- ✓ Distributed Cache
- ✓ Response Cache
- ✓ Cache Tag Helper

Use Cases

- Store frequently used data
- Reduce DB calls

Expiration

- Absolute expiration

- Sliding expiration

12. Session vs Cache (Final Clarity)

Feature	Session	Cache
Scope	Per-user	Application-wide
Storage	Server	Memory/Provider
Use Case	User state	Performance

13. Partial Views vs Layouts

Feature	Partial View	Layout
Purpose	Reusable UI fragment	Site template
RenderBody()	No	Yes
Used inside	Views/Layout	Views
Best for	Header/Footer/Menu	Page-level skeleton

14. Tag Helpers — Key Ones

- Anchor Tag Helper
- Input Tag Helper
- Image Tag Helper (asp-append-version)
- Form Tag Helper
- Environment Tag Helper

15. View Imports & Organization

_ViewImports.cshtml:

- Adds namespaces globally
- Enables Tag Helpers
- Prevents repetition

Hierarchy:

1. Local view
 2. Folder-level ViewImports
 3. Root ViewImports
-